Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

CODE VISUALIZER WITH STEP BY STEP ANIMATIONS

The domain of the Project:

Static/Dynamic Code Analysis & Algorithm Visualization

COURSE NAME

G36 Python

Team Mentors (and their designation):

Bhargavesh Dakka (Decision Scientist - II @ Mu Sigma Inc.)

Team Members:

Mr. V.Niteesh

Period of the project:

20 Days

March 2026 to April 2026

Declaration

The project titled "Code Visualizer with step by step animations" has been mentored by Bhargavesh Dakka sir, organised by SURE Trust, from April 2026 to August 2026, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Mr. V.Niteesh

Mentor's Name	Bhargavesh Dakka
Designation—Company Name	Decision Scientist - II @ Mu Sigma Inc.

Prof. Radhakumari
Executive Director & Founder
SURE Trust

Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion

Executive Summary

1. Project Objectives

The primary goal of CodeViz was to engineer a full-stack, deterministic execution engine that bridges the gap between static code writing and dynamic runtime understanding. By visualizing memory states and calculating code complexity in real-time, the project aimed to deliver:

- **Educational Algorithm Visualization:** Translate complex, invisible backend memory transitions into a high-speed, interactive frontend state machine.
- **Real-Time Code Profiling:** Provide instantaneous static analysis (code smells, cyclomatic complexity) and dynamic analysis (runtime execution profiling) for Python scripts.

2. Methods & Approach

To achieve a high-performance, stateless architecture, the system was built from scratch without relying on heavy frontend frameworks or machine learning models. The core technical approach included:

- **Static & Dynamic Analysis:** Utilized Python's native ast module to mathematically traverse code trees for complexity grading, alongside sys.settrace inside isolated subprocesses to capture line-by-line memory snapshots.
- **Zero-Dependency Frontend:** Engineered a custom Vanilla JavaScript rendering engine capable of parsing backend timeline arrays into sub-second CSS animations without virtual DOM overhead.
- **Stateless Integration:** Bypassed traditional database requirements by building a direct, client-side REST API integration with GitHub, allowing users to fetch raw .py files dynamically.

3. Key Findings & Architectural Successes

The development of CodeViz proved the viability of a highly lean, custom-built deterministic engine. Critical project highlights include:

- **Zero Database Overhead:** Utilizing GitHub as a live data source eliminated server storage costs and prevented data synchronization issues.
- **Secure Sandbox Execution:** Implementing strict 8-second backend timeouts and isolated subprocesses successfully mitigated the risks of infinite loops and malicious runtime crashes.
- **Seamless State Decoupling:** Decoupling the backend JSON timeline generation from the frontend visual projector ensured the browser never crashes, even when processing high-complexity algorithm loops.

4. Strategic Recommendations (Future Scaling)

Based on the current architecture, we recommend the following technical roadmap to transition CodeViz from a local prototype to a globally scalable application:

1. **WebAssembly Migration:** Transition the Python execution engine from the FastAPI backend directly to the user's browser using Pyodide (WebAssembly). This will drop server compute costs to absolute zero and allow infinite concurrent users.
2. **Containerized Security:** If maintaining server-side execution, wrap the Python subprocesses in strictly partitioned, network-isolated Docker containers to achieve enterprise-grade security.
3. **Algorithm Expansion:** Scale the frontend heuristic detection engine to automatically identify and animate advanced algorithms, such as Merge Sort and Quick Sort, by extending the regex priority queue.

5. Conclusion & Business Impact

CodeViz successfully positions itself at the cutting edge of Developer Tooling and EdTech. By choosing a deterministic, framework-free architecture, the project delivered a lightning-fast, highly educational debugging suite with zero database costs and a deeply scalable foundation.

Introduction

Background and Context of the Project

In the domains of computer science education and software engineering, understanding the flow of an algorithm requires a deep mental model of how a machine allocates and mutates memory over time. Historically, bridging the gap between writing static code and understanding dynamic execution has been a major friction point. Standard Integrated Development Environments (IDEs) offer debuggers, but these are often too heavy, complex, or unintuitive for rapid conceptual learning. Furthermore, as the tech industry increasingly relies on probabilistic AI models (like LLMs) to explain code, there is a growing risk of "hallucinations" or inaccurate explanations. This creates a strong need for lightweight, highly accessible, and **deterministic** tools that can visually prove exactly how code is behaving at the compiler and memory levels.

Problem Statement and Goals

Problem Statement: Developers and students lack an accessible, lightweight visualization tool that can instantly translate raw Python code into an intuitive, frame-by-frame visual state machine. Existing solutions either require heavy local software installations, lack real-time algorithmic animations, or fail to provide concurrent static analysis (like code complexity metrics).

Project Goals: The primary objective of the CodeViz project is to engineer a web-based, full-stack execution engine capable of:

- **Static Analysis:** Utilizing mathematical code parsing to instantly grade code health, nesting depth, and cyclomatic complexity without running the script.
- **Dynamic Execution:** Capturing memory states, variable mutations, and standard outputs line-by-line during runtime.
- **Algorithm Visualization:** Translating the backend execution data into a high-speed, interactive frontend animation.
- **Frictionless Access:** Bypassing the need for user accounts or databases by fetching code directly from public GitHub repositories.

Scope and Limitations

Scope of the Project:

- The application is strictly designed to parse and execute **Python 3** code.
- The system supports native Python data structures (lists, dictionaries, sets) and built-in algorithms.
- The architecture includes a FastAPI backend for execution routing, a custom Python tracer for memory hooking, and a Vanilla JavaScript frontend for DOM manipulation and animation.

- Integration is limited to the public GitHub REST API for fetching remote .py files.

Limitations of the Project:

- **External Libraries:** The execution engine operates in a secure, sandboxed environment. It does not support heavy third-party libraries (such as numpy, pandas, or tensorflow) to maintain rapid execution speeds.
- **Execution Timeouts:** To prevent server crashes from infinite loops (e.g., while True:), a strict 8-second execution timeout is enforced via the backend subprocess.
- **Payload Capping:** To prevent the browser from crashing due to massive memory arrays, the backend tracer is capped at a maximum of 400 execution steps per request.
- **Statelessness:** The platform does not utilize a persistent database; therefore, user code edits are not saved between sessions unless pushed to their own GitHub repository.

Innovation Component

The CodeViz platform introduces several innovative engineering patterns that separate it from standard CRUD (Create, Read, Update, Delete) applications:

- **Deterministic Engine vs. Probabilistic AI:** Rather than relying on AI to guess how an algorithm works, CodeViz utilizes Python's native C-level debugging hooks (sys.settrace) and Abstract Syntax Trees (ast.NodeVisitor) to guarantee 100% mathematical accuracy of the execution timeline.
- **Database-Free Architecture:** By building a client-side integration with the public GitHub API and rewriting standard URLs into raw content streams, the project innovatively uses GitHub as its real-time database, dropping storage costs to zero.
- **Framework-Free Rendering:** While modern web development heavily relies on frameworks like React or Angular, CodeViz innovates by using a custom Vanilla JavaScript state machine. This eliminates Virtual DOM overhead, allowing the browser to render and animate hundreds of HTML elements in sub-second intervals for real-time visualization.

Project Objectives

Core Objectives and Goals

The overarching goal of the CodeViz project is to demystify software execution by bridging the gap between writing static code and understanding dynamic machine behavior. To achieve this, the project targeted four specific technical and educational goals:

- **Algorithmic Transparency:** To build an educational tool that visually proves exactly how an algorithm mutates memory, variable by variable, in real-time, moving away from "black-box" execution.
- **Deterministic Accuracy:** To engineer a mathematical, deterministic backend that relies on Python's native C-level hooks rather than probabilistic AI models, ensuring 100% accuracy in execution profiling.
- **Stateless Architecture:** To design a highly cost-effective, zero-database infrastructure that utilizes the public GitHub REST API as a real-time, dynamic code repository.
- **High-Performance Rendering:** To achieve sub-second, frame-by-frame UI animations by bypassing heavy frontend frameworks (like React) in favor of a raw, Vanilla JavaScript state machine that directly manipulates the browser's Document Object Model (DOM).

Expected Outcomes

By successfully implementing the core objectives, the project is expected to yield the following outcomes:

- **An Enhanced Learning Paradigm:** Users will be able to instantly paste complex logic (such as nested loops or sorting algorithms) and watch a visual flipbook of the memory states, significantly reducing the learning curve for data structures and algorithms.
- **Secure Code Sandboxing:** The platform will successfully intercept and neutralize fatal runtime crashes, infinite loops, and malicious code through isolated subprocesses and strict execution timeouts, ensuring the host server remains stable.
- **Zero Storage Overhead:** The application will operate at maximum cost-efficiency by leveraging a client-side fetch protocol, eliminating the need to store, sync, or secure user source code in a proprietary database.

Key Deliverables

The project culminates in the delivery of a fully integrated, full-stack web application consisting of the following core modules:

- **Static Analysis Module (parser.py):** An Abstract Syntax Tree (AST) traversal engine that calculates cyclomatic complexity, nesting depth, and detects code smells without executing the script.

- **Dynamic Tracing Engine (tracer.py):** A backend Python profiler that utilizes sys.settrace to capture local variables, memory states, and standard outputs line-by-line during runtime.
- **Vanilla JS Rendering Engine (index.html/js):** A custom-built frontend UI that acts as a finite state machine, translating the backend JSON timeline into colored, animated HTML elements.
- **GitHub Integration Pipeline:** A client-side JavaScript module that translates standard GitHub repository URLs into raw content streams via the GitHub REST API.

Methodology and Results

Methods and Technology Used

The CodeViz platform was developed using a decoupled, full-stack architecture prioritizing execution speed and zero-dependency environments. The core methods and technologies include:

- **Frontend Rendering Engine:** Built entirely with **Vanilla JavaScript (ES6+), HTML5, and CSS3**. Instead of relying on Virtual DOM frameworks like React, the UI operates as a Custom Finite State Machine that directly manipulates the browser's Document Object Model (DOM) to achieve sub-second animation rendering.
- **Backend Execution Engine:** Powered by **Python 3**. The system utilizes Python's native **ast (Abstract Syntax Tree)** module for static mathematical parsing, and **sys.settrace** C-level hooks for dynamic memory profiling.
- **Network Communication:** Operates on standard **RESTful HTTP methods** (POST, GET) with strict CORS middleware implementation to allow cross-origin resource sharing between the client browser and the server.
- **Algorithmic Heuristics:** Uses **Regular Expressions (Regex)** for pattern matching to auto-detect algorithm types (e.g., sorting vs. searching) and to parse standard error (stderr) crash reports.

Tools and Software Used

- **FastAPI:** A high-performance Python web framework used to route backend requests and enforce payload validation.
- **Uvicorn:** An ASGI web server implementation used to host the FastAPI application.
- **Subprocess Module:** Python's built-in process creation tool, used to spin up isolated, sandboxed blast chambers for running untrusted user code safely.
- **GitHub REST API:** Specifically the Git Trees API and Raw Content API, utilized as a serverless database to fetch remote .py files.

Data Collection Approach

Because CodeViz is designed as a strict **stateless application**, it does not utilize a traditional persistent database (like PostgreSQL or MongoDB) for data collection. Instead, data is dynamically ingested through two distinct pipelines:

1. **Direct Client Input:** Code authored locally by the user is ingested via the frontend text editor array and transmitted to the backend as a strictly typed JSON payload.
2. **Remote API Ingestion:** When a user provides a GitHub URL, a client-side API integration rips the URL into its owner and repository components. It then hits the public GitHub REST API to fetch the repository tree, filters for Python files, and dynamically pulls the raw source code into the execution environment. GitHub acts as the application's single source of truth.

Project Architecture

The architecture is decoupled into three primary tiers: the Client Tier, the API Tier, and the Analysis Tier.

- **The Client Tier (Vanilla JS Frontend):** Acts as the orchestrator. It intercepts user inputs, manages the GitHub fetching process, and packages the code into HTTP POST requests. Once the backend responds, the JavaScript State Machine parses the JSON timeline and injects colored HTML tags to animate variable states frame-by-frame.
- **The API Tier (FastAPI Gateway):** Receives the HTTP requests and acts as the bouncer. It validates the payload structure, throws HTTP 400 or 422 errors for malformed requests, and routes the code to the appropriate analysis engine.
- **The Analysis Tier (Backend Engines):**
    - **Static Analyzer (parser.py):** Converts the raw text string into a mathematical syntax tree. It recursively walks the nodes to count decision points (if/for/while loops), extracts documentation strings, and calculates the overall Cyclomatic Complexity metric without running the code.
    - **Dynamic Sandbox (tracer.py):** The FastAPI server spawns an isolated subprocess. Inside this process, sys.settrace attaches a spy function to the Python compiler. As the code executes, the tracer captures the local variable dictionary at every single line, packages it into a chronological array, and returns it to the API layer. This process is heavily guarded by an 8-second runtime timeout and a 400-step memory limit to prevent infinite loops and payload overflow.

Final Project Working Screenshots

Note: Insert your actual project screenshots below. I have written professional captions for you to pair with them.

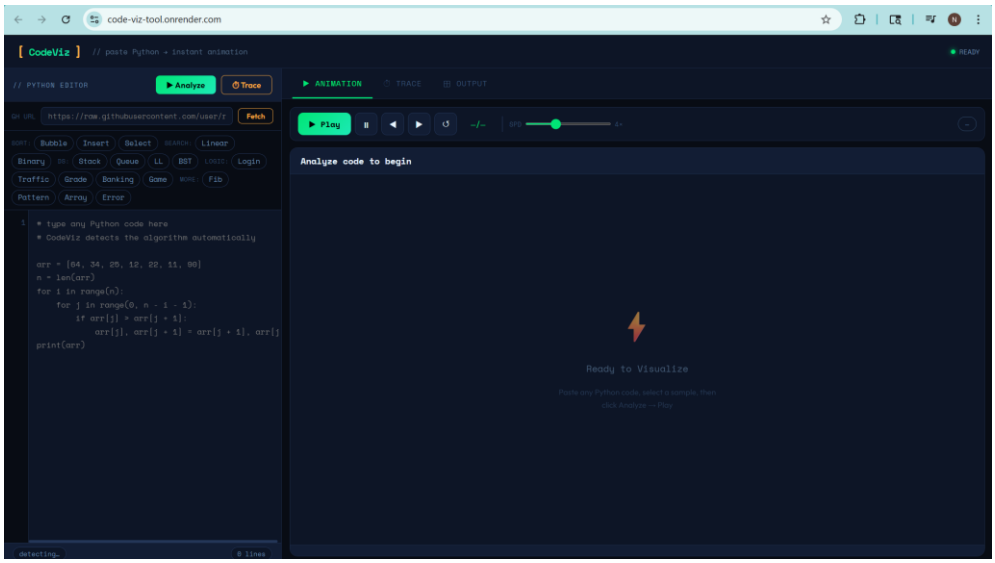

Figure 1: The Main CodeViz Interface. The integrated text editor allows users to input raw Python logic or fetch remote files, providing immediate visual feedback and error highlighting.

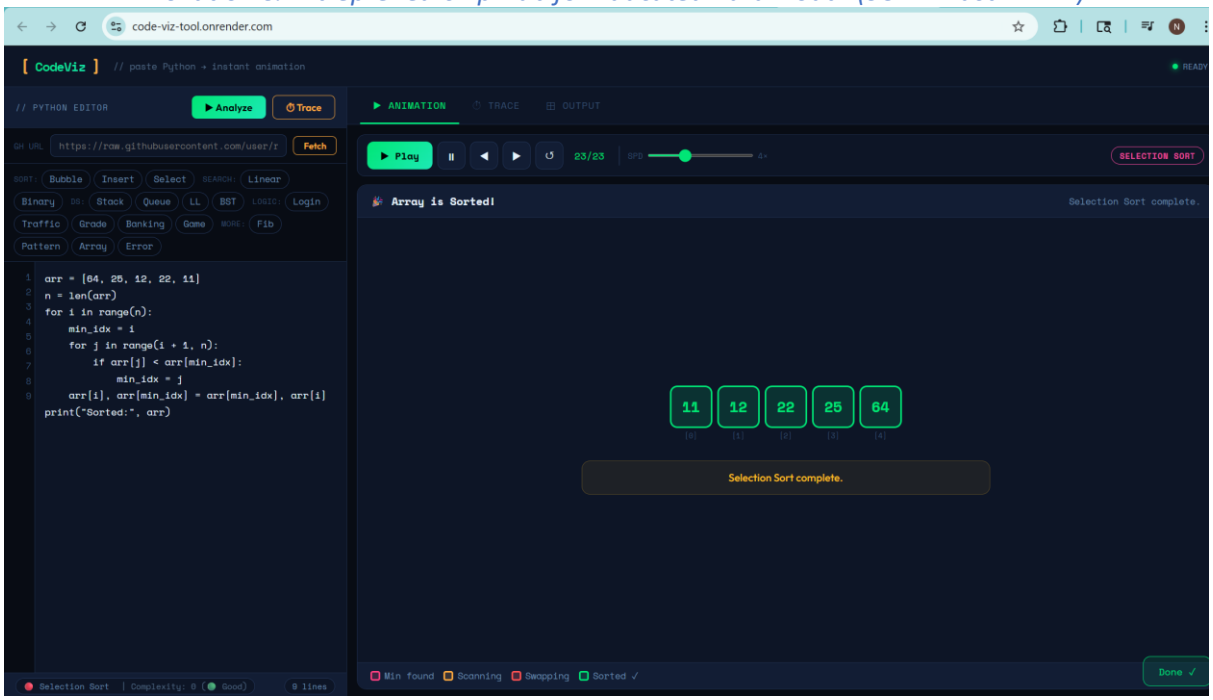

Figure 2: Real-Time Algorithm Visualization. The frontend Vanilla JavaScript state machine successfully translating the backend memory timeline into an interactive, color-coded animation of an active algorithm.

Figure 3: AST Static Analysis Output. The system successfully computing and displaying the Cyclomatic Complexity score, nesting depth, and code health metrics prior to runtime.

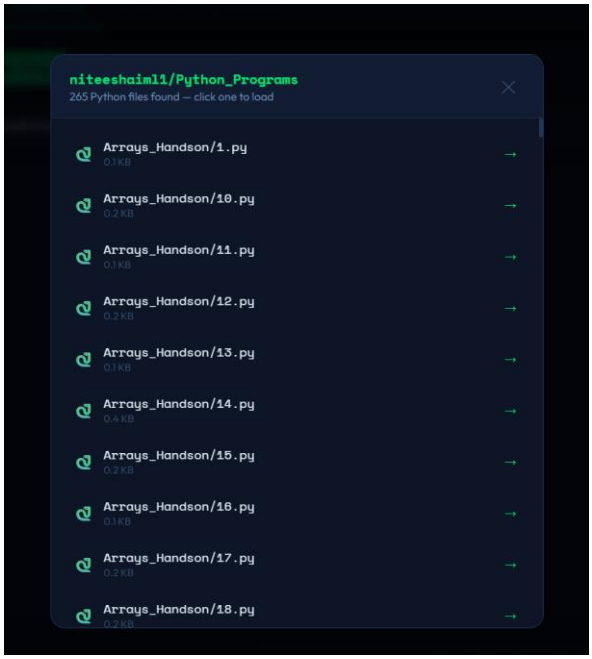

Figure 4: Client-Side GitHub API Integration. The custom modal interface successfully fetching and filtering a remote GitHub repository tree, allowing the user to select and download raw Python files dynamically.

Project GitHub Link

The complete source code, architectural documentation, and deployment instructions can be found at the following repository:

Learning and Reflection

1. Technical Learnings (Engineering & Architecture)

Building CodeViz from the ground up required moving beyond basic web development and diving deep into compiler-level architecture and system optimization. Key technical takeaways include:

- **Mastery of AST and Compiler Internals:** I gained a deep understanding of how Python reads and parses code natively. By utilizing the ast.NodeVisitor class, I learned how to translate raw syntax into a mathematical tree to build a custom Static Analyzer, completely demystifying how enterprise tools like SonarQube work.
- **Dynamic Memory Hooking:** Learning to use Python's C-level sys.settrace function was a major breakthrough. I learned how to intercept a running compiler line-by-line, extract local variable memory states, and package them into a deterministic timeline.
- **Vanilla JS State Machines & DOM Manipulation:** By choosing not to use a heavy framework like React, I learned how to build a high-performance Finite State Machine using pure JavaScript. I mastered direct DOM manipulation to render sub-second UI animations, proving that sometimes Vanilla JS is significantly faster and leaner than Virtual DOM architectures.
- **Secure Subprocess Sandboxing:** Running untrusted user code poses a massive security risk. I learned how to implement strict execution timeouts and capture standard error (stderr) streams using Python's subprocess module to create a secure blast chamber that protects the host server from crashing.
- **Stateless API Integrations:** I learned how to successfully leverage the public GitHub REST API to fetch raw code, allowing me to build a zero-database application that drastically reduces deployment complexity and server costs.

2. Management and Product Learnings

Developing this platform also provided significant insights into product management, architectural decision-making, and scope control:

- **Ruthless Prioritization (Scope Control):** Initially, the temptation was to build a tool that supported every programming language and every complex library (like Pandas or NumPy). I learned that defining strict scope constraints—focusing purely on standard Python 3 for educational algorithms—was vital to shipping a high-quality, bug-free prototype on time.
- **Decoupled Architecture Design:** I learned the immense value of separating concerns. By keeping the Python execution engine completely agnostic and decoupled from the JavaScript visualization frontend, I ensured that a bug in the UI animation would never crash the backend server, and vice-versa.
- **Designing for the User Experience (UX):** Translating raw backend JSON data into a digestible frontend format taught me a lot about educational UX. I learned that simply throwing a

crash report at a user is bad design; catching that crash with Regex and translating it into a "Teacher Hint" dramatically improves the learning experience.

3. Overall Experience Summary

My overall experience building CodeViz was incredibly challenging but deeply rewarding. It bridged the gap between being a standard web developer and thinking like a Systems Architect.

The most fulfilling moment of the project was successfully mapping a backend memory array to a frontend CSS animation for the first time—watching an abstract algorithm physically "come to life" on the screen proved that the underlying math and architecture were sound. This project solidified my passion for Developer Tooling and Educational Technology, and it has given me the confidence to tackle low-level systems programming, secure cloud deployments, and complex state management in future roles.

Conclusion and Future Scope

Conclusion: Objectives and Achievements Recap

The CodeViz project successfully bridged the critical gap between static code authoring and dynamic execution understanding. By engineering a deterministic, full-stack visualization engine, the project achieved its primary objective of making complex algorithmic behavior transparent and accessible to learners and developers.

Rather than relying on probabilistic AI models, CodeViz proved the efficacy of a strictly mathematical approach. The successful implementation of Python's ast module provided instantaneous code health metrics, while the sys.settrace engine accurately captured line-by-line memory mutations. Furthermore, the project achieved significant architectural efficiency. By completely decoupling the frontend Vanilla JavaScript state machine from the backend Python subprocesses, the application renders complex, high-speed UI animations without the overhead of heavy frameworks. Finally, the strategic integration of the public GitHub REST API eliminated the need for a proprietary database, resulting in a highly secure, stateless, and cost-effective deployment footprint. CodeViz stands as a robust proof-of-concept for the future of lightweight, deterministic educational tooling.

Future Scope of the Project

While the current architecture provides a highly functional baseline, the project has immense potential for future scaling and feature expansion. The following strategic milestones have been identified for the next phases of development:

- **WebAssembly (Pyodide) Integration:** The most significant future upgrade involves migrating the backend Python execution directly into the user's browser using WebAssembly. This would eliminate the need for server-side Python processing entirely, dropping cloud compute costs to zero and allowing the application to support infinite concurrent users without backend bottlenecking.
- **Enterprise-Grade Containerization:** If server-side execution is maintained, the current subprocess blast chambers will be upgraded to fully isolated, read-only Docker containers. This will provide an enterprise-grade security layer, ensuring absolute protection against malicious system-level commands.
- **Expanded Algorithmic Heuristics:** The frontend regex detection engine will be expanded to automatically recognize and animate a wider variety of advanced algorithms, including graph traversals (like Dijkstra's or A* Search) and complex divide-and-conquer methods (like Merge Sort).
- **Multi-Language Support:** While currently optimized for Python 3, the underlying architectural pattern is language-agnostic. The future scope includes building parallel backend parsers to support JavaScript (via Node.js memory profiling) and Java (via JVM TI hooks), expanding the platform's reach to a wider demographic of software engineers.